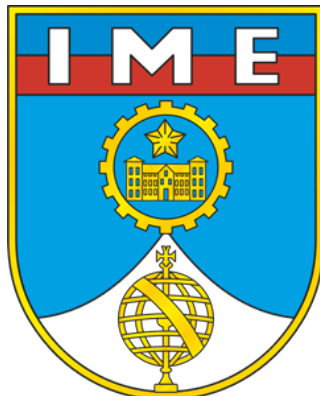


**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA**



**SEÇÃO DE ENGENHARIA ELÉTRICA (SE/3)
PROGRAMAÇÃO APLICADA A ENGENHARIA ELÉTRICA**

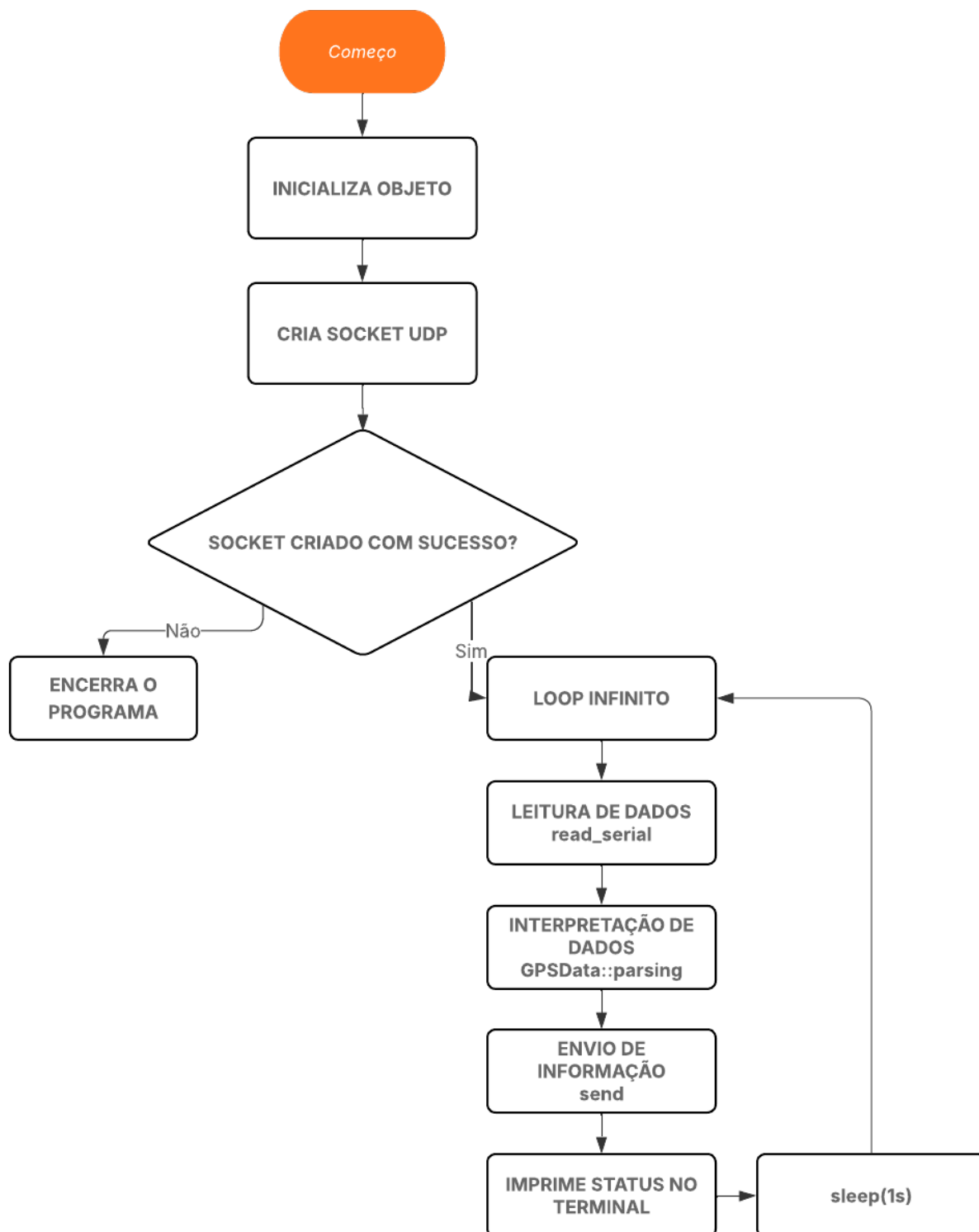
RELATÓRIO FINAL

PROFESSOR: TEN NICOLAS OLIVEIRA

**COMPONENTES DO GRUPO:
MATHEUS DEYVISSON DE OLIVEIRA MORENO PINTO
EDUARDO DE AMARIZ GALVÃO
DANIEL MACHADO BARBOSA**

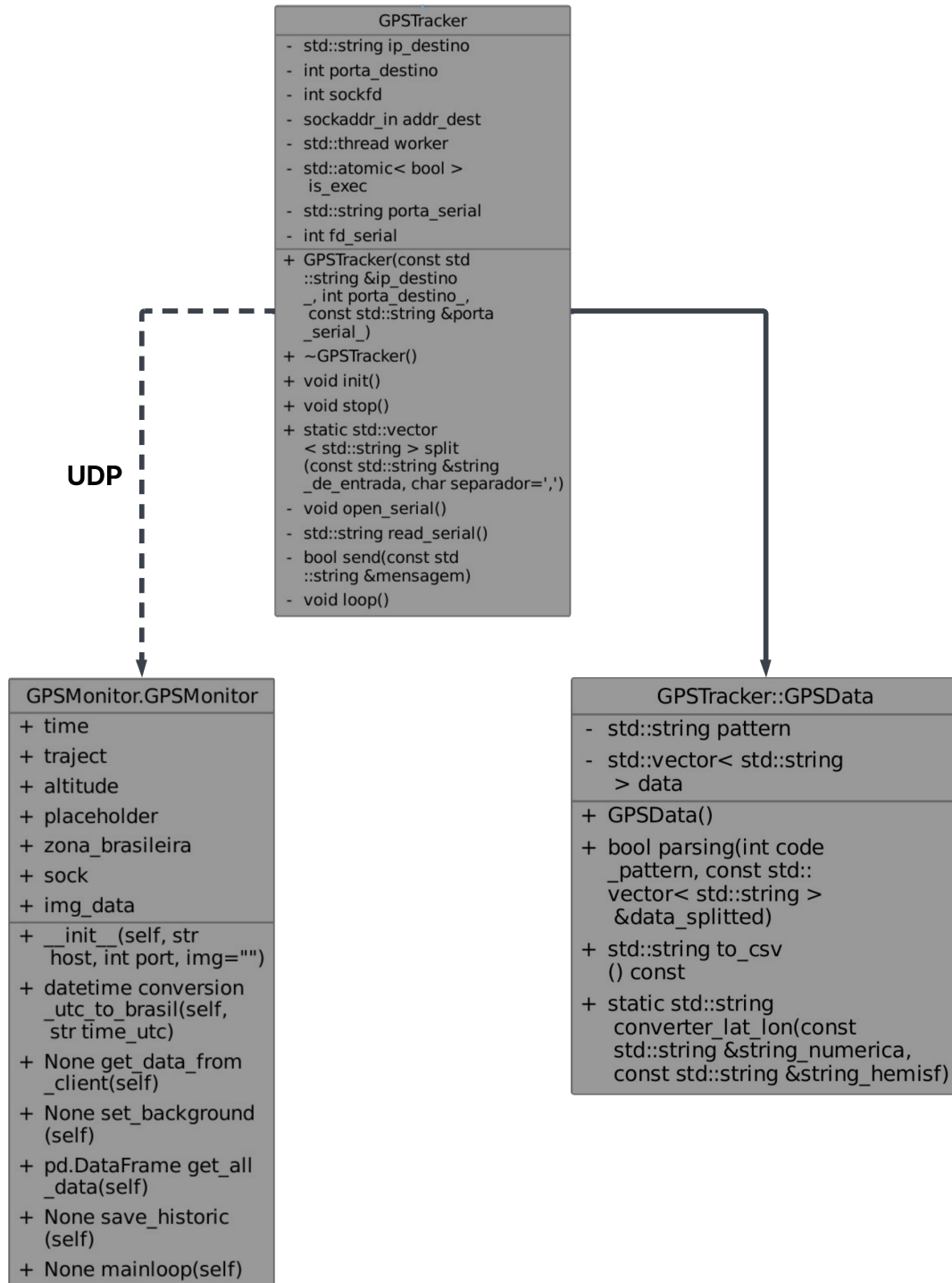
**RIO DE JANEIRO
11 DE NOVEMBRO DE 2025**

1 Fluxograma



2 Diagrama de classes

A seguir tem-se o diagrama de classes do projeto, onde GPSTracker possui a responsabilidade de obter os dados do sensor, interpreta-los e enviar essas informações via socket UDP. Para isso, essa classe utiliza o GPSTracker para representar os dados vindos do GPS. Por conseguinte, a classe GPSTracker fica responsável por apresentar esses dados recebidos em tempo real na interface.



3 Instruções de compilação e uso

Para ser válida a compilação:

Configuração do Ambiente de Desenvolvimento

Além do compilador padrão para verificações superficiais, será necessário um compilador específico para o microcontrolador que estamos focando.

- **Download do Compilador Cross-Platform**

Verifique o *link* dado pelo professor e realize:

```
tar -xvf arm-buildroot-linux-gnueabihf_sdk-buildroot.tar.gz
```

Isso descompactará o kit de desenvolvimento da placa, permitindo diversas funcionalidades.

Warning: Não se preocupe com o tempo que demorará.

Ao realizar a descompactação, você terá:

```
arm-buildroot-linux-gnueabihf_sdk-buildroot/  
|-- bin/  
|   |-- arm-buildroot-linux-gnueabihf-gcc  
|   |-- arm-buildroot-linux-gnueabihf-g++  
|   ...  
|-- arm-buildroot-linux-gnueabihf/  
|-- sysroot  
|   ...
```

Observe na pasta `bin` a presença dos compiladores `gcc` e `g++`.

Makefile

Caso fôssemos utilizar apenas o compilador, precisaríamos escrever comandos grandes e fáceis de errar. Sendo assim, faz-se necessário um intermediário:

```
sudo apt install make
```

Doxygen

Utilizaremos esse software para criar documentações de forma automática. **Não é necessário que todos os contribuintes possuam essa ferramenta**, entretanto, é estritamente obrigatório a utilização do mesmo padrão de documentação.

```
sudo apt install doxygen graphviz
```

```
sudo apt install texlive-latex-extra texlive-fonts-recommended texlive-fonts-extra texlive-fonts-extra
```

O segundo download faz referência à construção do arquivo `.pdf`.

Acesso à Placa

Utilizaremos o protocolo SSH para acessá-la remotamente. Naturalmente, será necessário um cabo Ethernet e que os IPs estejam na mesma faixa. É interessante que você utilize a função `ping` para verificar se estão conectadas corretamente.

Assim que a conexão for confirmada, dentro do PowerShell, execute:

```
ssh root@192.168.42.2
```

Será solicitada a senha e, posteriormente, o acesso será permitido.

Para enviar arquivos para a placa, volte novamente ao PowerShell e execute:

```
scp -o GPSTracker root@192.168.42.2:/
```

Assim, o arquivo estará no mesmo local que as pastas `root` do sistema Linux. Além disso, pode ser necessário dar permissão de execução ao binário. Para tanto:

```
chmod +x GPSTracker
```

GPSTracker

O arquivo binário **GPSTracker** presente neste diretório já representa o executável a ser inserido na placa para completa operacionalidade.

Com a adição da feature de envio via rede, torna-se necessária a entrada de novos argumentos: o *IP* e a *porta de destino*.

Sendo assim, a execução fica, por exemplo:

```
./GPSTracker 127.0.0.1 1234
```

make build

Compilará a aplicação utilizando as flags necessárias e o compilador específico, gerando um executável de pronto uso no módulo.

make debug IP=... PORTA=...

Compilará, executará e, posteriormente, apagará uma simulação para debug, na qual um módulo GPS virtual envia dados seriais para nosso interpretador, permitindo que possamos analisar *insights* e simular a realidade.

make docs

Para contribuintes, gerará um PDF contendo a documentação da aplicação geral.

make showup

Apresentará uma interface inteligente referente aos dados do sensor, mostrando:

- Dados em tempo real;
- Histórico de Dados;
- Salvamento de Histórico de Dados.

Necessário ambiente virtual e bibliotecas *streamlit* e *streamlit_autorefresh*.

4 Documentação do projeto, do código e do protocolo

4.1 Documentação do projeto

O objetivo do projeto é desenvolver uma solução embarcada usando o kit de desenvolvimento **STM32MP1 DK1** e o sensor **GY-GPS6MV2**, a fim de monitorar de forma remota e contínua qualquer tentativa de violação e/ou comprometimento de uma carga.

4.2 Documentação do código

Em primeira instância, criou-se a classe interpretadora chamada GPSTracker, cujas funcionalidades são:

- Ler dados do sensor;
- Interpretar os dados lidos;
- Enviar as informações.

Ademais, vale ressaltar o fluxo de funcionamento dos códigos.

1. Inicialização: a classe configura um socket UDP para transmissão dos dados processados e uma porta serial, na qual o sensor enviará os dados;
2. Leitura de dados: O método "read_serial" coleta os dados diretamente da porta serial;
3. Interpretação dos dados: A classe GPSTData é completamente responsável pelo *parsing* dos dados, conseguindo traduzir mensagens no estilo GPGBA, a partir da qual pode-se obter, com segurança, informações de horário em UTC, latitude, longitude e altitude.
4. Envio de informações: A função "send" envia as informações via socket UDP para determinada máquina e porta.

4.3 Documentação do protocolo de comunicação

Para a comunicação cliente e servidor, utilizou-se o protocolo UDP (User Datagram Protocol), além disso, usou-se a porta 5000. A classe interpretadora envia mensagens no seguinte padrão:

TIME_IN_UTC,LATITUDE,LONGITUDE,ALTITUDE

Exemplo:

```
164918.00,-22.956000,-43.166000,32.0  
165027.00,22.955500,43.165500,32.0  
165358.00,-22.956000,-43.166000,-32.0
```

A mensagem final haverá sempre, no mínimo, 33 bytes. Conforme alguma coordenada seja negativa, isso acrescentará os bytes correspondentes aos caracteres "-", sem que haja perda de precisão, isso é:

- TIME_IN_UTC: sempre terá 9 bytes destinados a si, dado que não há horário negativo.
- LATITUDE: sempre terá no mínimo 9 bytes destinados, podendo haver mais um para o sinal de menos, mantendo a mesma precisão.
- LONGITUDE: sempre terá no mínimo 9 bytes destinados, podendo haver mais um para o sinal de menos, mantendo a mesma precisão.
- ALTITUDE: sempre terá no mínimo 3 bytes destinados, podendo haver mais um para o sinal de menos, mantendo a mesma precisão.

Não há *checksum* nem cabeçalho de tamanho, entretanto, caso se deseje adicionar esses requisitos, basta analisar a função "send()" do GPSTracker.

5 Descrição do sensor e funcionamento



Figura 1: Imagem representativa do módulo GPS GY-GPS6MV2.

Módulo GPS GY-GPS6MV2

O módulo **GY-GPS6MV2** é um receptor de sinal de posicionamento global baseado no chip **Ublox NEO-6M**, amplamente utilizado em projetos embarcados e aplicações de rastreamento por sua confiabilidade, precisão e baixo consumo energético.

Esse módulo é responsável por receber sinais de satélites da constelação GPS (Global Positioning System), processando as informações de tempo e posição para determinar coordenadas geográficas em tempo real. Através desses dados, é possível obter latitude, longitude, altitude, velocidade e horário universal (UTC).

Componentes principais:

- **Chipset NEO-6M:** Responsável pela recepção e decodificação dos sinais dos satélites GPS, com suporte a até 22 canais simultâneos.
- **Antena cerâmica integrada:** Permite a captação de sinais de alta sensibilidade, dispensando antenas externas em ambientes com boa visibilidade do céu.
- **Memória EEPROM:** Armazena dados de configuração e o último estado do módulo, agilizando o tempo de inicialização (*Time To First Fix*).
- **LED indicador:** Pisca quando o módulo obtém fixação de satélites (lock GPS), indicando operação normal.
- **Interface UART (TX/RX):** Responsável pela comunicação serial com microcontroladores ou computadores, transmitindo dados no formato **NMEA 0183**.

Funcionamento:

O módulo inicia a varredura de sinais GPS ao ser energizado, captando dados transmitidos por múltiplos satélites. Através de cálculos de triangulação, baseados no tempo de propagação dos sinais, o módulo determina sua posição tridimensional (latitude, longitude e altitude).

Esses dados são enviados ao microcontrolador por meio da interface serial, utilizando sentenças NMEA, que seguem um padrão de comunicação textual. Exemplo de uma sentença típica:

\$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

O microcontrolador, por sua vez, interpreta essas informações e as utiliza em aplicações como rastreamento de veículos, navegação, registro de rotas e sistemas embarcados de localização.

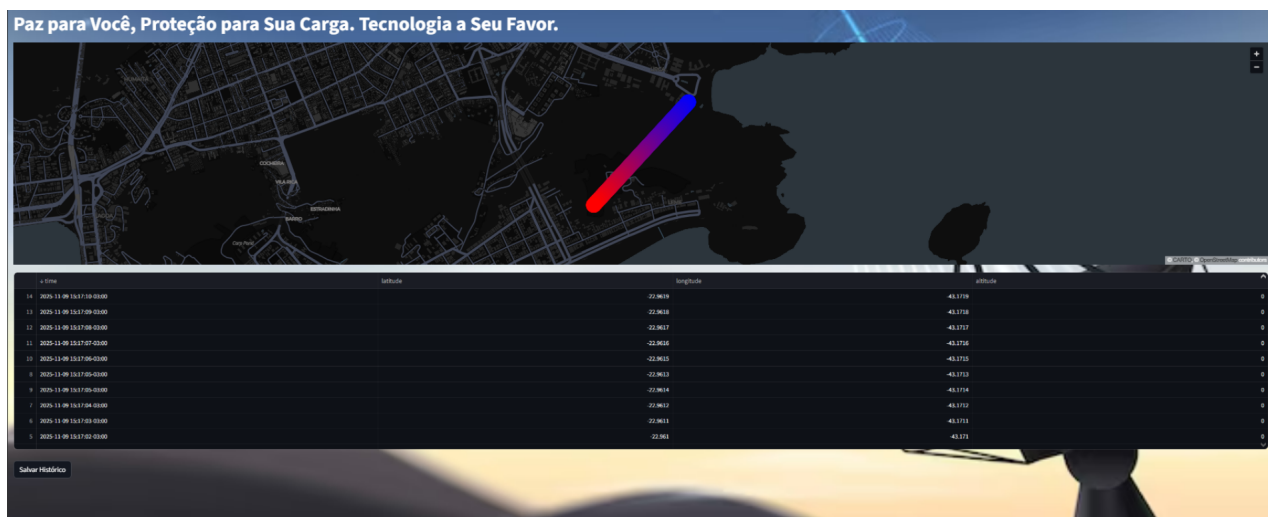
Características técnicas principais:

- Alimentação: 3,3V a 5V;
- Corrente típica: 45mA;
- Baud rate padrão: 9600 bps (configurável);
- Comunicação: UART (TX/RX);
- Frequência de atualização: até 5 Hz;
- Sensibilidade: até -161 dBm;
- Protocolo de saída: NMEA 0183.

Em suma, o **GY-GPS6MV2** oferece uma solução compacta e eficiente para aquisição de coordenadas geográficas precisas, sendo ideal para projetos de rastreamento, drones, robótica e automação embarcada.

6 Capturas de tela da interface

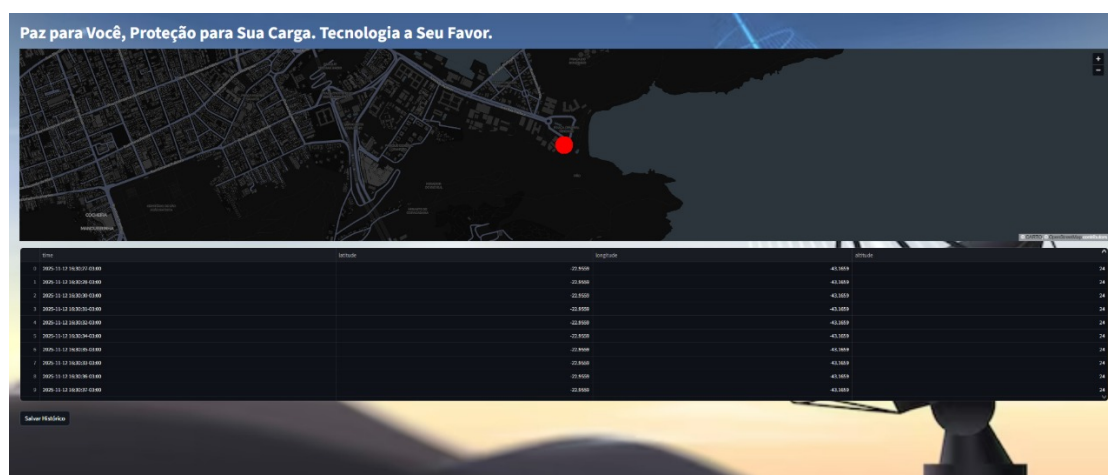
A seguir encontra-se a captura de tela da interface. Vale ressaltar que os pontos mais recentes do GPS estão em vermelho enquanto que os mais antigos encontram-se em azul. Além disso, a imagem foi ligeiramente esticada nas laterais para completa visualização dos dados.



7 Análise dos valores medidos

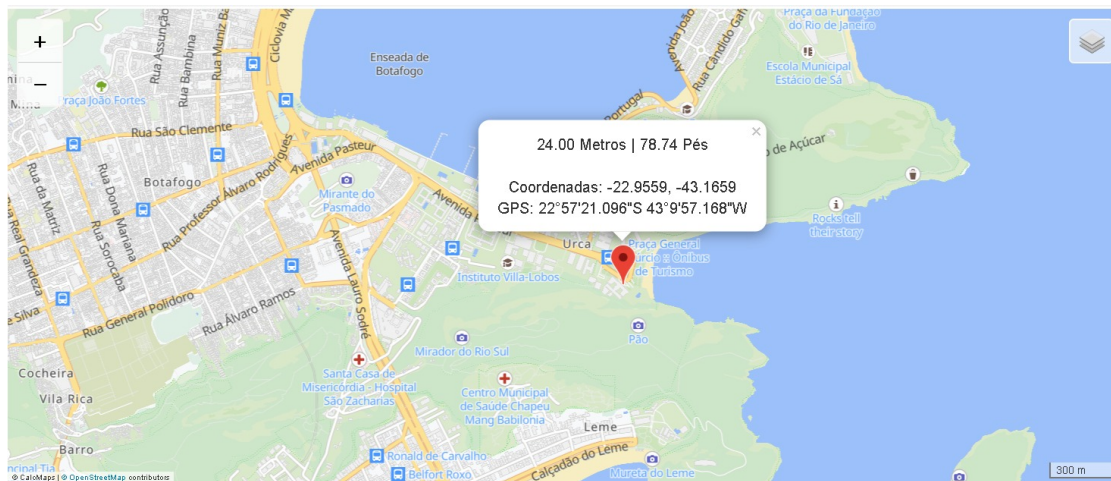
Para analisar a veracidade dos valores medidos, comparou-se o valor obtido pelo GPS com o valor real obtido no site <https://www.calcmaps.com/pt/map-elevation/>.

Dado obtido pelo GPS:



- Latitude: -22,9559
- Longitude: -43,1659
- Altitude: 24

Dado real:



Perceba que os pontos coincidem, validando, portanto, os valores medidos pelo GPS.